

04-代码生成与优化复习讲义

目录

1	04 代码生成与优化复习讲义
---	----------------

2

1 04 代码生成与优化复习讲义

对应资料:

- Slides/Slides/Ch8-1-Target Code Generation.pdf
- Slides/Slides/Ch8-2-Translation out of SSA.pdf
- Slides/Slides/Ch8-3-Register Allocation.pdf
- Slides/Slides/Ch10-Instruction Scheduling.pdf
- Review/compile.pdf
- Exam/2025.pdf 指令调度题
- Exam/过时/2020-final-exam.pdf 目标代码生成题型
- Exam/过时/2023-exam-A.pdf 目标代码生成题型

1.1 一、目标代码生成

目标代码生成把 IR 转换为目标机器指令。核心任务:

- 指令选择。
- 寄存器分配。
- 指令调度。
- 调用约定和栈帧处理。
- 地址计算和内存访问。

1.2 二、寻址模式

常见模式:

LD R1, a(R2) : 取地址 $(R2)+a$ 处数据放入 R1
LD R1, 100(R2) : 取地址 $(R2)+100$ 处数据放入 R1
LD R1, *R2 : 取地址 $R2$ 处数据放入 R1
LD R1, *100(R2) : 先取 $(R2)+100$ 处数据作为新地址, 再取该地址数据
LD R1, #100 : 将立即数 100 放入 R1

1.3 三、块内优化

常见优化：

- 局部公共子表达式消除。
- 强度削弱。
- 使用机器习语。
- 去除冗余 LOAD/STORE。
- 窥孔优化。

例：

```
x * 2      -> x + x
x = x + 1  -> INC x
t1 = a+b; t2 = a+b -> t2 = t1
```

1.4 四、从 SSA 转出

phi 消除思路：

```
a = phi(a1 from P1, a2 from P2)
```

改写：

```
P1 末尾插入 a = a1
P2 末尾插入 a = a2
删除 phi
```

完整流程要处理并行复制：

1. 把 phi 的输入复制放到对应前驱块末尾。
2. 多个复制要按并行赋值语义处理。
3. 存在环形复制时引入临时变量。
4. 删除多余复制。

1.5 五、寄存器分配

目标：

无限虚拟寄存器 $\rightarrow$ 有限物理寄存器

约束：

- 生成正确代码。
- 使用不超过 k 个物理寄存器。
- 减少 spill 的 load/store。
- 控制栈空间。

1.6 六、局部寄存器分配

基本块内没有控制流分叉，可计算变量活跃区间。

MAXLIVE：某个程序点之后仍活跃的变量数。

若 $\text{MAXLIVE} > k$ ，要 spill。选择 spill 的经验规则：

选择下一次使用距离最远的变量 spill。

1.7 七、全局寄存器分配与图染色

冲突图：

- 顶点：变量或虚拟寄存器。
- 边：两个变量活跃区间相交，不能分配同一物理寄存器。

k -染色：

- 若冲突图可 k -染色，则可用 k 个寄存器完成分配。
- 若不可染色，要 spill。

Chaitin 算法：

1. 找度数小于 k 的点，删除并压栈。

2. 重复直到图空。
3. 若找不到度数小于 k 的点，选一个点 spill。
4. 出栈时分配颜色。

Chaitin-Briggs:

- 度数大于等于 k 的点先不直接 spill，也压栈。
- 出栈染色失败时再 spill。

1.8 八、指令调度

目的:

- 把相互独立的指令放近，让硬件流水线和乱序执行利用并行性。
- 减少停顿。
- 提高硬件利用率。

依赖类型:

True dependence / RAW : 写后读。 i 写 x , j 读 x , i 必须在 j 前。
Anti dependence / WAR : 读后写。 i 读 x , j 写 x , j 不能提前破坏 i 读到的值。
Output dependence / WAW: 写后写。 i 写 x , j 写 x , 二者顺序影响最终值。

寄存器重命名:

- 可消除 Anti dependence 和 Output dependence。
- 不能消除 True dependence。

1.9 九、列表调度

步骤:

1. 建依赖图，节点是指令，边是依赖。
2. 计算每个节点的就绪条件。
3. 每个周期从 ready list 中选可发射指令。

4. 使用优先级打破并列，例如关键路径长度更大者优先。
5. 得到满足依赖的拓扑序。

考场写法：

依赖图 -> ready list -> 每周期选择 -> 新指令序列

1.10 十、自测题与答案

1.10.1 题 1：寄存器重命名能消除哪类依赖？

答案：能消除 Anti dependence 和 Output dependence，不能消除 True dependence。

1.10.2 题 2：冲突图中一条边表示什么？

答案：两个变量活跃区间重叠，不能放入同一个物理寄存器。

1.10.3 题 3：局部寄存器分配中何时 spill？

答案：某程序点活跃变量数超过可用寄存器数时要 spill。